

21MHC305J – Manufacturing

Processes

LLJ - II Report

SEMESTER VI

Name :

Registration Number :



Department of Mechatronics Engineering
SRM Institute of Science and Technology,
Kattankulathur – 603203

APRIL – 2026

Monocular Metrology System using Homography

1. Problem Statement

The objective of this project is to develop a high-accuracy monocular vision-based metrology system that can estimate real-world dimensions of objects using a single camera. The system must compensate for perspective distortion and object height variations to improve measurement accuracy.

2. Methodology

The system follows a structured pipeline:

- Camera Calibration using chessboard images
- Homography estimation between image plane and real-world plane
- Perspective transformation of selected points
- Distance computation in real-world coordinates
- Height compensation using geometric scaling

3. Mathematical Model

Homography Transformation:

$$x' = Hx$$

$x \rightarrow$ image coordinate (pixels)

$H \rightarrow$ Homography Matrix

$x' \rightarrow$ real world coordinate (cm)

Distance Calculation:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

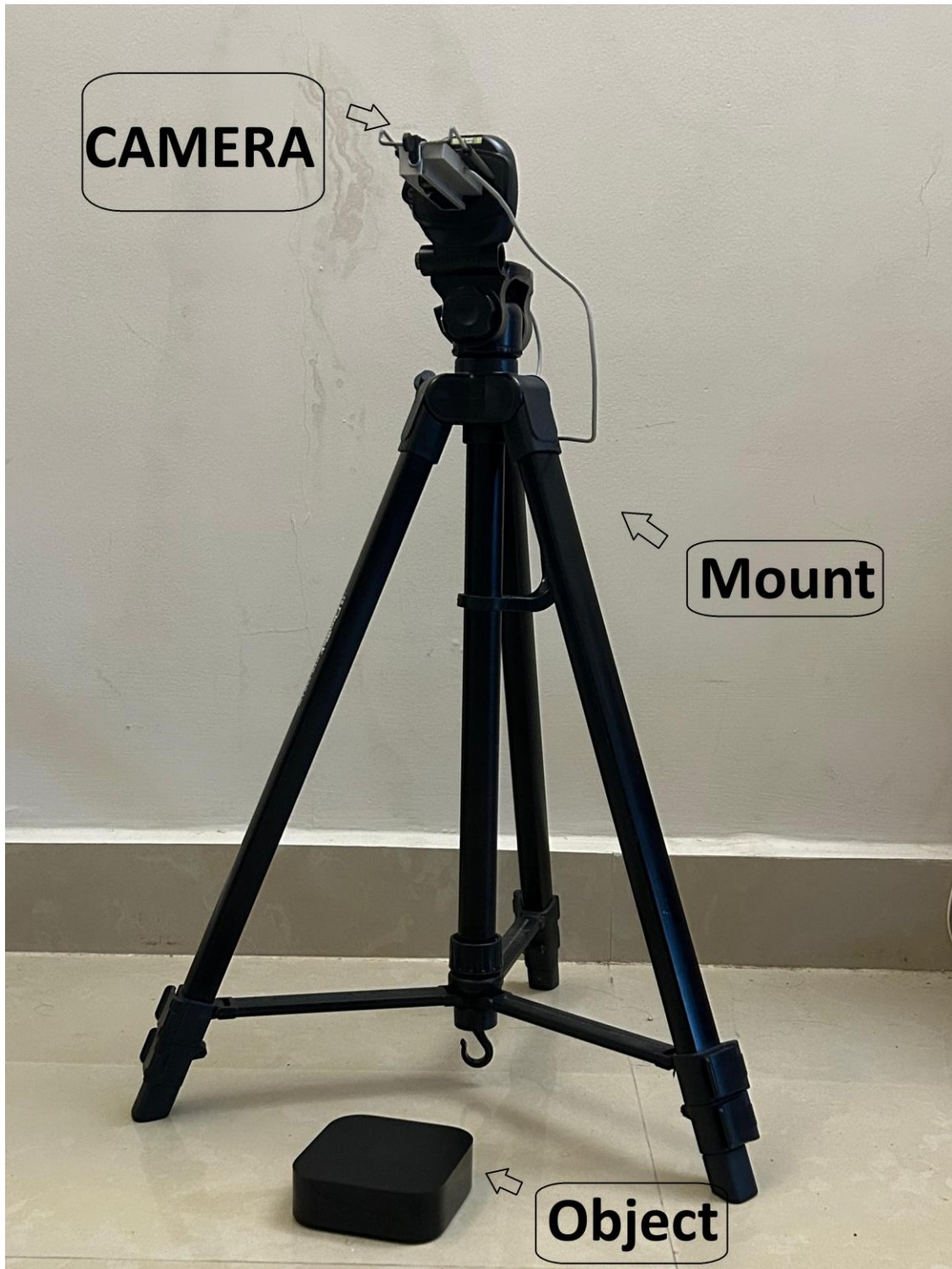
d is the distance in cm

Height Compensation:

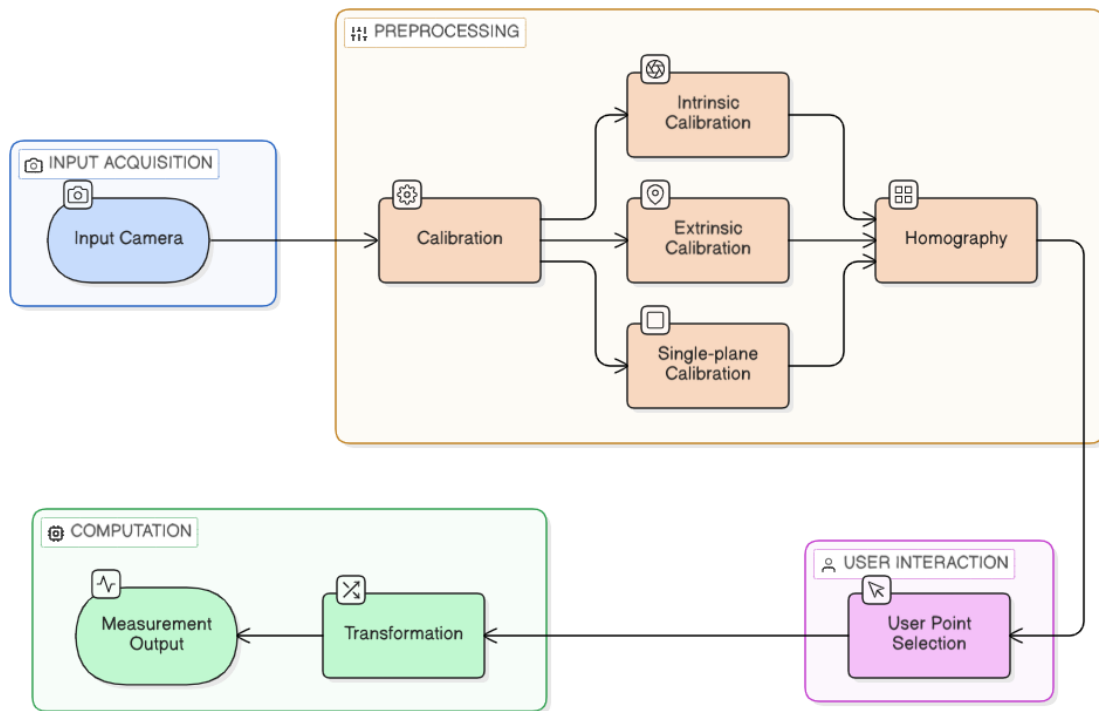
$$d_{corrected} = d / (H_c / (H_c - H_o))$$

Where H_c = camera height, H_o = object height. This corrects perspective scaling error.

4. Experimental Setup



5. System Architecture



1. Input Acquisition

The system begins with an input camera that captures real-time images of the scene. This serves as the primary data source for all subsequent processing.

- Provides raw image frames
- Ensures fixed resolution for consistency
- Acts as the base for calibration and measurement

2. Preprocessing (Calibration & Homography)

This is the most critical stage where the system establishes the mapping between image space and real-world space.

a. Calibration

The calibration block determines camera parameters required for accurate measurement.

- **Intrinsic Calibration**
Computes internal camera parameters such as focal length, principal point, and distortion coefficients.

- **Extrinsic Calibration**

Determines the camera's position and orientation relative to the world coordinate system.

- **Single-Plane Calibration**

Defines a reference plane (working surface) on which measurements are based.

b. Homography Estimation

Using the calibrated parameters, a homography matrix is computed.

- Maps 2D image points → real-world plane coordinates
- Enables perspective correction
- Eliminates distortion caused by camera angle

This transformation is essential for converting pixel distances into real-world units (cm).

3. User Interaction

The system allows manual point selection for measurement.

- User selects two points on the object
- These points act as measurement endpoints
- Ensures flexibility for different object shapes

This step bridges human input with automated computation.

4. Computation (Transformation & Measurement)

a. Transformation

Selected image points are transformed using the homography matrix.

- Converts pixel coordinates into real-world coordinates
- Accounts for perspective distortion

b. Measurement Output

The system computes the Euclidean distance between transformed points.

- Distance is displayed in centimeters
- Includes **height compensation correction** for improved accuracy

6. Working

6.1 Image Capturing

The following snippet of code helps in the capturing of images and saving them in a folder labelled “calib_images” these images help in the calibration of the camera and finding the intrinsic and extrinsic parameters of the same.

The intrinsic parameters of a camera depend on how it captures the images. Parameters such as focal length, aperture, field-of-view, resolution, etc. govern the intrinsic matrix of a camera model.

The extrinsic parameters of a camera depend on its location and orientation.

```
calib_capture.py 1
C: > Users > Namit > Desktop > metrology > calib_capture.py > start_camera
1  import cv2
2  import os
3  def start_camera():
4      cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)
5      if not cap.isOpened():
6          print("Error: Cannot open camera")
7          return
8
9      # Set resolution
10     cap.set(3, 640)
11     cap.set(4, 480)
12
13     # Create folder
14     save_path = "calib_images"
15     os.makedirs(save_path, exist_ok=True)
16
17     img_count = 0
18
19     print("Press 's' to save image | 'q' to quit")
20
21     while True:
22         ret, frame = cap.read()
23
24         if not ret:
25             print("Failed to grab frame")
26             break
27
28         cv2.imshow("Camera", frame)
29
30         key = cv2.waitKey(1) & 0xFF
31
32         if key == ord('s'):
33             filename = f"{save_path}/img_{img_count}.png"
34             cv2.imwrite(filename, frame)
35             print(f"Saved: {filename}")
36             img_count += 1
37
38         elif key == ord('q'):
39             break
40
41     cap.release()
42     cv2.destroyAllWindows()
43 if __name__ == "__main__":
44     start_camera()
```

6.2 Camera Calibration

Once the calibration images have been captured, the following code is run, to determine the above mentioned intrinsic and extrinsic parameters of the camera.

These parameters help in offsetting any error that could have been possible due to manufacturing defects.

The image capturing for the calibration for the camera only needs to be done once given that the camera remains same.

```
calibration.py 1
C: > Users > Namit > Desktop > metrology > calibration.py > ...
1  import cv2
2  import numpy as np
3  import glob
4
5  # Chessboard dimensions (INNER corners)
6  chessboard_size = (8, 5)
7
8  # Prepare object points (real world coordinates)
9  objp = np.zeros((chessboard_size[0] * chessboard_size[1], 3), np.float32)
10 objp[:, :2] = np.mgrid[0:8, 0:5].T.reshape(-1, 2)
11
12 objpoints = [] # 3D points
13 imgpoints = [] # 2D points
14
15 # Load calibration images
16 images = glob.glob('calib_images/*.png')
17
18 print(f"Found {len(images)} images")
19
20 for fname in images:
21     img = cv2.imread(fname)
22     gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
23
24     ret, corners = cv2.findChessboardCorners(gray, chessboard_size, None)
25
26     if ret:
27         objpoints.append(objp)
28         imgpoints.append(corners)
29
30         # Draw corners (for verification)
31         cv2.drawChessboardCorners(img, chessboard_size, corners, ret)
32         cv2.imshow('Corners', img)
33         cv2.waitKey(200)
34     else:
35         print(f"Chessboard NOT detected in {fname}")
36
37 cv2.destroyAllWindows()
38
39 # Calibration
40 ret, K, dist, rvecs, tvecs = cv2.calibrateCamera(
41     objpoints, imgpoints, gray.shape[::-1], None, None
42 )
43 print("\n=== Calibration Results ===")
44 print("Camera Matrix (K):\n", K)
45 print("\nDistortion Coefficients:\n", dist)
```

6.3 Plane Calibration

This code helps in establishing a relative transformation between the image plane and the actual world plane i.e. the actual picture is measured in terms of pixel but the real-world objects are measured in terms of mm.

Hence, to derive this relation the plane calibration is done. This code is also supposed to be run once only but the height of the tripod (on which the camera is mounted) can vary between to separated measurements hence the code is run once before every measurement.

```
1. calibrate_plane_big.py 1 X
C: > Users > Namit > Desktop > metrology > 1. calibrate_plane_big.py > ...
1  import cv2
2  import numpy as np
3
4  points_img = []
5  captured = False
6
7  # camera resolution
8  orig_w = 640
9  orig_h = 480
10
11 # much larger display window
12 disp_w = 1600
13 disp_h = 1200
14
15 scale_x = orig_w / disp_w
16 scale_y = orig_h / disp_h
17
18 # === REAL WORLD COORDINATES (cm) ===
19 points_real = np.array([
20     [0, 0],
21     [10, 0],
22     [10, 10],
23     [0, 10]
24 ], dtype="float32")
25
26
27 def click_event(event, x, y, flags, param):
28     global points_img, img_display
29
30     if not captured:
31         return
32
33     # convert display mouse coordinates to original pixel coordinates
34     x_real = int(x * scale_x)
35     y_real = int(y * scale_y)
36
37     if event == cv2.EVENT_LBUTTONDOWN:
38
39         points_img.append([x_real, y_real])
40         print(f"Clicked: {x_real}, {y_real}")
41
42         cv2.circle(img_display, (x_real, y_real), 6, (0, 0, 255), -1)
43
44         if len(points_img) == 4:
```



```

27 def click_event(event, x, y, flags, param):
45     pts_img = np.array(points_img, dtype="float32")
46     H, _ = cv2.findHomography(pts_img, points_real)
47     print("\nHomography Matrix:\n", H)
48     np.save("homography.npy", H)
49     print("Saved as homography.npy")
50 # CAMERA
51 cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)
52 cap.set(3, orig_w)
53 cap.set(4, orig_h)
54 cv2.namedWindow("Calibration", cv2.WINDOW_NORMAL)
55 cv2.resizeWindow("Calibration", disp_w, disp_h)
56 cv2.setMouseCallback("Calibration", click_event)
57 while True:
58     if not captured:
59         ret, frame = cap.read()
60         if not ret:
61             break
62         display = frame.copy()
63         cv2.putText(display, "Press 'c' to capture",
64                     (20,40), cv2.FONT_HERSHEY_SIMPLEX,
65                     1.0, (0,255,0), 2)
66         display = cv2.resize(display, (disp_w, disp_h))
67         cv2.imshow("Calibration", display)
68     else:
69         display = cv2.resize(img_display, (disp_w, disp_h))
70         cv2.imshow("Calibration", display)
71         key = cv2.waitKey(1)
72         if key == ord('c') and not captured:
73             img_display = frame.copy()
74             captured = True
75             print("Image captured. Now click 4 points.")
76         if key == ord('r'):
77             captured = False
78             points_img.clear()
79             print("Reset")
80         if key == ord('q'):
81             break
82 cap.release()
83 cv2.destroyAllWindows()

```

6.4 Live Measurement

After the calibrations are done the live measurements can be done. In this code the user is given the option to click an image first and once the image is captured the user can select any two points on the image and the user will get a distance between those two points.

If the object has any height to it the user can provide a rough estimate of the height and the code will provide a length while offsetting the variation in height.

```
2.Live_mes.py 1 X
C: > Users > Namit > Desktop > metrology > 2.Live_mes.py > ...
1  import cv2
2  import numpy as np
3
4  # Load homography
5  H = np.load("homography.npy")
6
7  points = []
8  captured = False
9  mouse_pos = (0,0)
10
11 # original camera resolution
12 orig_w = 640
13 orig_h = 480
14
15 # display window size
16 disp_w = 1000
17 disp_h = 750
18
19 scale_x = orig_w / disp_w
20 scale_y = orig_h / disp_h
21
22
23 def click_event(event, x, y, flags, param):
24     global points, img_display, mouse_pos
25
26     mouse_pos = (x,y)
27
28     if not captured:
29         return
30
31     # convert mouse coordinate → real image coordinate
32     x_real = int(x * scale_x)
33     y_real = int(y * scale_y)
34
35     if event == cv2.EVENT_LBUTTONDOWN:
36
37         points.append([x_real, y_real])
38
39         # smaller blue dot
40         cv2.circle(img_display, (x_real, y_real), 3, (255,0,0), -1)
41
42         if len(points) == 2:
43
44             pts = np.array(points, dtype="float32").reshape(-1,1,2)
45             real_pts = cv2.perspectiveTransform(pts, H)
```

```

p1 = real_pts[0][0]
p2 = real_pts[1][0]

dist = np.linalg.norm(p1 - p2)

print(f"Distance: {dist:.2f} cm")

pt1 = tuple(map(int, points[0]))
pt2 = tuple(map(int, points[1]))

cv2.line(img_display, pt1, pt2, (255,0,0), 2)

mid = ((np.array(pt1)+np.array(pt2))/2).astype(int)

cv2.putText(img_display, f"{dist:.2f} cm",
            tuple(mid), cv2.FONT_HERSHEY_SIMPLEX,
            0.7, (255,0,0), 2)

points = []

```

```

# CAMERA
cap = cv2.VideoCapture(1, cv2.CAP_DSHOW)
cap.set(3, orig_w)
cap.set(4, orig_h)

cv2.namedWindow("Measurement", cv2.WINDOW_NORMAL)
cv2.resizeWindow("Measurement", disp_w, disp_h)

cv2.setMouseCallback("Measurement", click_event)

while True:
    if not captured:
        ret, frame = cap.read()
        if not ret:
            break

        display = frame.copy()

        cv2.putText(display, "Press 'c' to capture",
                    (20,30), cv2.FONT_HERSHEY_SIMPLEX,
                    0.7, (0,255,0), 2)

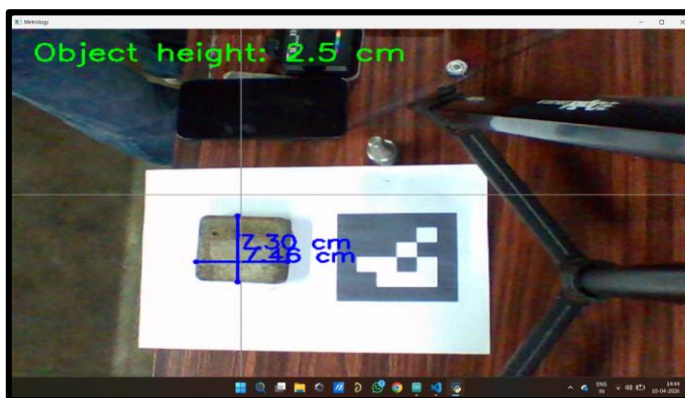
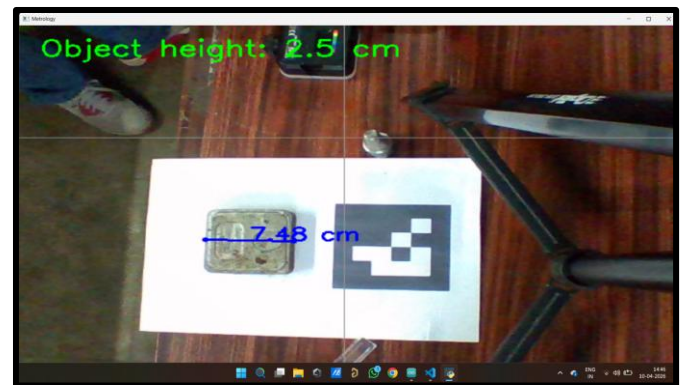
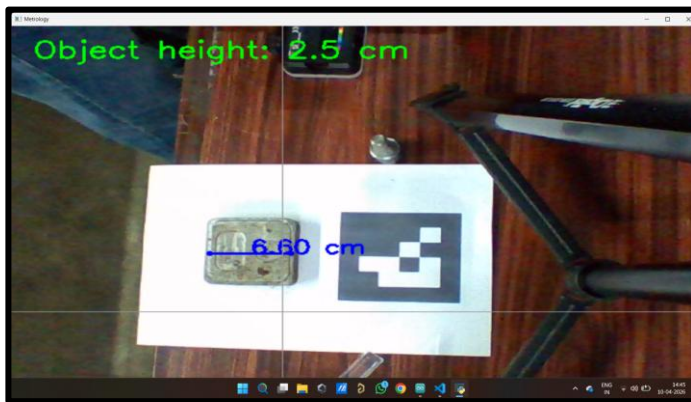
```

```

91
92     display = cv2.resize(display, (disp_w, disp_h))
93     cv2.imshow("Measurement", display)
94 else:
95     preview = img_display.copy()
96     # convert mouse to image coordinates
97     mx, my = mouse_pos
98     mx_real = int(mx * scale_x)
99     my_real = int(my * scale_y)
100    # draw crosshair
101    cv2.line(preview, (mx_real,0), (mx_real,orig_h), (150,150,150), 1)
102    cv2.line(preview, (0,my_real), (orig_w,my_real), (150,150,150), 1)
103    # live distance preview
104    if len(points) == 1:
105        pt1 = np.array(points[0], dtype="float32").reshape(-1,1,2)
106        pt2 = np.array([[mx_real,my_real]], dtype="float32").reshape(-1,1,2)
107        real_pt1 = cv2.perspectiveTransform(pt1, H)[0][0]
108        real_pt2 = cv2.perspectiveTransform(pt2, H)[0][0]
109        dist = np.linalg.norm(real_pt1 - real_pt2)
110        pt1_draw = tuple(map(int, points[0]))
111        pt2_draw = (mx_real, my_real)
112        cv2.line(preview, pt1_draw, pt2_draw, (0,255,255), 1)
113        mid = ((np.array(pt1_draw)+np.array(pt2_draw))//2).astype(int)
114        cv2.putText(preview, f"{dist:.2f} cm",
115                    tuple(mid), cv2.FONT_HERSHEY_SIMPLEX,
116                    0.7, (0,255,255), 2)
117
118    display = cv2.resize(preview, (disp_w, disp_h))
119    cv2.imshow("Measurement", display)
120    key = cv2.waitKey(1)
121    if key == ord('c') and not captured:
122        img_display = frame.copy()
123        captured = True
124        print("Image captured. Click first point.")
125    if key == ord('r'):
126        captured = False
127        points.clear()
128        print("Reset")
129    if key == ord('q'):
130        break
131
132cap.release()
133cv2.destroyAllWindows()

```

7. Input & Output Screens



8. Results & Analysis

Object	Measured (cm)	Actual (cm)
Phone	14.54	14.5
Metal Block(inner)	6.6	6.6
Metal Block	7.48	7.5
Gear	4.55	4.6

The system demonstrates consistent measurement accuracy across varying object heights. Minor variations are due to manual point selection and camera noise.

Keeping all the results in mind the relative error the system is demonstrating is

$$-0.06\text{mm} < e < +0.02\text{mm}$$

9. Advantages

- No need for expensive sensors
- Works with a single camera
- Real-time measurement capability
- Height compensation improves accuracy significantly

10. Limitations

- Sensitive to calibration accuracy
- Manual point selection introduces error
- Limited height range (0–3 cm in this setup)

10. Conclusion

The project successfully demonstrates a robust monocular metrology system using homography and geometric correction. The integration of height compensation significantly enhances real-world usability.